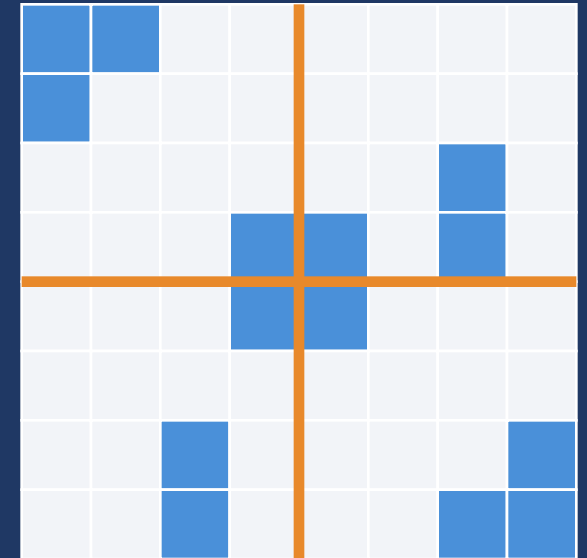
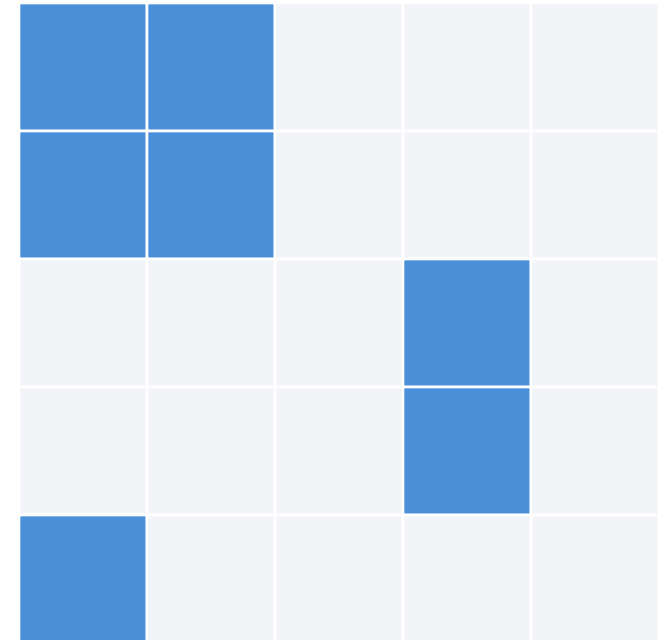

Sistemas Operacionais 2026/II - Prof. Filipo Novo Mor - PUCRS



O problema

Contar componentes de 1s com conectividade 8

- Matriz binaria: 0 = fundo, 1 = primeiro plano
- Objeto = celulas 1 ligadas por lado OU canto (8 vizinhos)
- Exemplo 1 (5x5): 3 objetos
- Desafio paralelo: um objeto pode atravessar as divisoes da matriz - somar contagens locais NAO basta



8 vizinhos: (-1,-1) (-1,0) (-1,1) (0,-1) (0,1) (1,-1) (1,0) (1,1)

Estrategia escolhida

Uma referencia sequencial + uma versao paralela equivalente

- Sequencial: flood fill ITERATIVO (pilha explicita) - referencia de correcao e de tempo
- Paralela: Pthreads (memoria compartilhada, sem copiar a matriz)
 - Matriz dividida em grade BL x BC de blocos (configuravel)
 - Fila dinamica de blocos: threads pegam o proximo bloco livre
 - Fase 1 (paralela): rotular componentes locais de cada bloco
 - Fase 2 (paralela): verificar fronteiras e unir rotulos (Union-Find)
- Objetos = soma das contagens locais - unioes efetivas
- ANSI C89, -Wall -Wextra -pedantic sem avisos, Linux e macOS

Versao sequencial

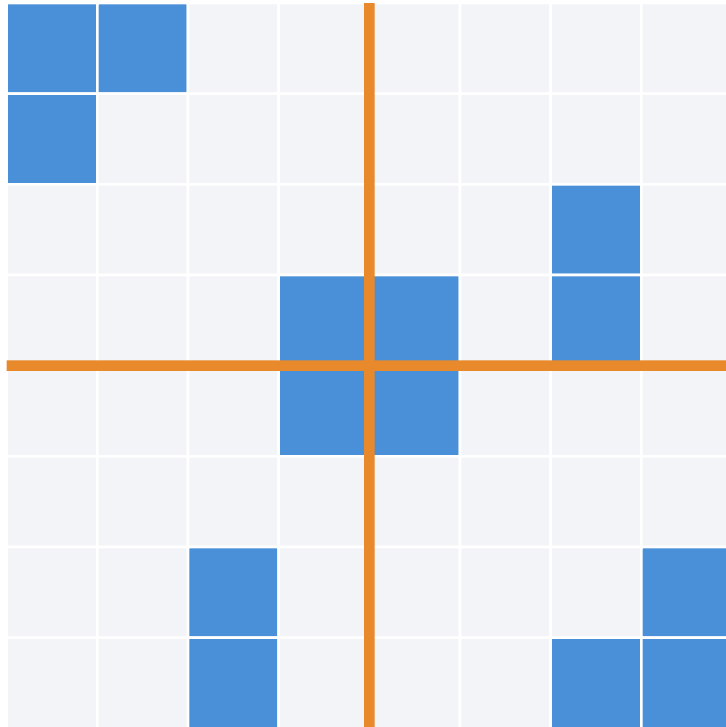
conta-objetos-sequencial.c

```
para cada celula i (ordem de linhas):  
    se M[i]==1 e nao visitado[i]:  
        objetos++  
        visitado[i]=1; empilha(i)  
        enquanto pilha nao vazia:  
            cel = desempilha()  
            para cada um dos 8 vizinhos v:  
                se M[v]==1 e nao visitado[v]:  
                    visitado[v]=1; empilha(v)
```

- Tempo $O(L \times C)$: cada celula entra na pilha no maximo 1 vez
- Marca ao EMPILHAR: sem duplicatas
- Sem recursao: pilha no heap cresce com realloc (recursao estouraria a stack em objetos grandes)
- Resultado das 5 matrizes = valores esperados

Decomposicao e fila dinamica

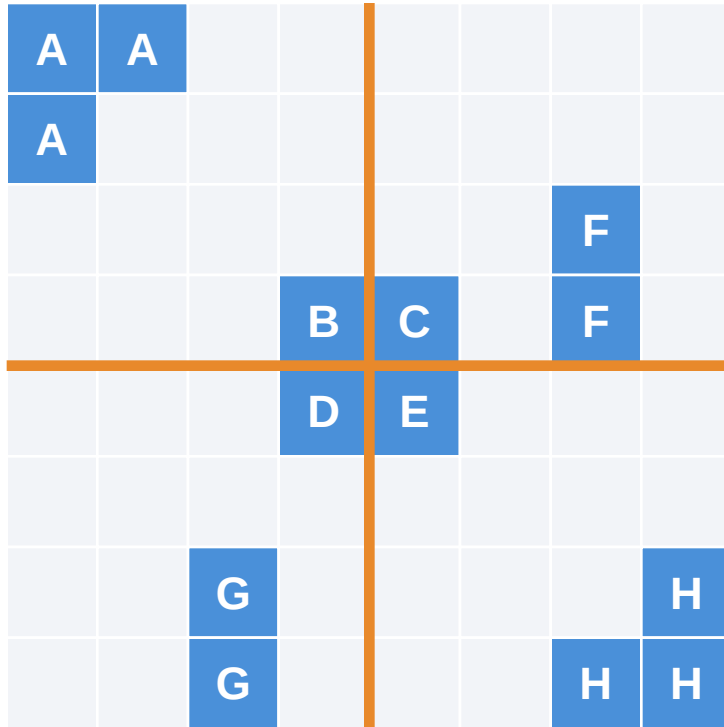
`./conta-objetos-paralelo arquivo <threads> [BL BC]`



- Bloco (i,j): linhas $[i*L/BL, (i+1)*L/BL)$ - sobras distribuidas
- Reproduz as grades 2x2 e 3x3 do enunciado
- Mais blocos que threads: fila dinamica balanceia a carga
- Fila = indice 'proximo' protegido por `mtx_fila`
- Cada celula pertence a UM bloco: escrita em `rotulo[]` sem lock

Fase 1 - rotulos locais (paralela)

Flood fill restrito aos limites do bloco



- Rotulo = indice da celula-semente + 1 ($I * C + c + 1$)
- Unico na matriz inteira sem contador global nem lock
- Cada thread tem a sua propria pilha
- Exemplo 3: 8 componentes locais (A..H)
- Objeto central B,C,D,E aparece 4 vezes!
- `pthread_join` = barreira antes da fase 2 (macOS nao tem `pthread_barrier`)

Sincronizacao e regioes criticas

Recurso	Risco	Protecao
proximo (fila)	2 threads pegam o mesmo bloco	mtx_fila
pai[] + unioes	corrida no find/union, uniao perdida	mtx_uf
rotulo[]	escrita simultanea	blocos disjuntos + join
matriz	-	somente leitura
fase1 -> fase2	ler rotulos incompletos	pthread_join

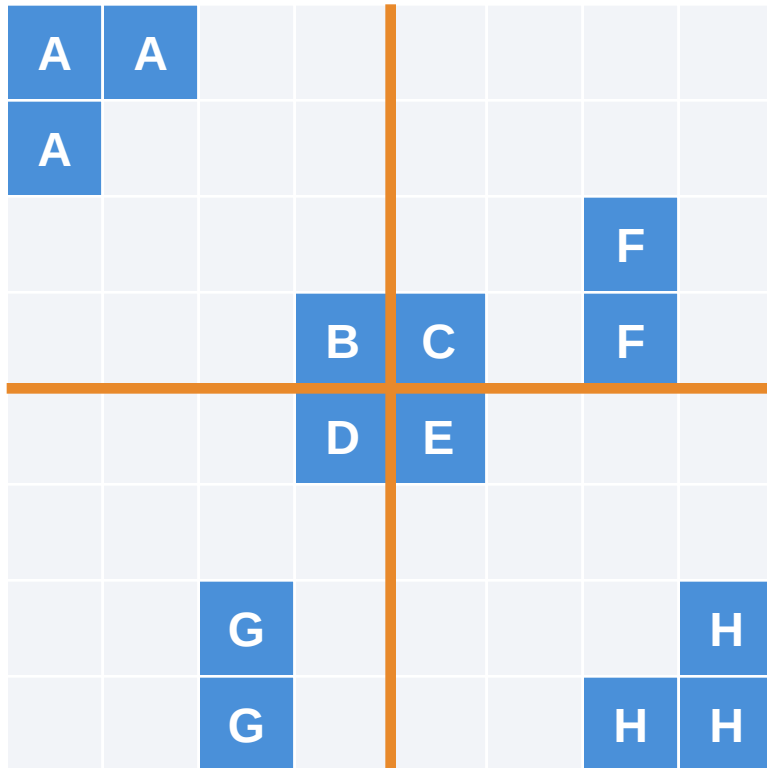
- Sem deadlock: nenhuma thread segura 2 mutexes ao mesmo tempo (sem espera circular)
- ThreadSanitizer: nenhuma data race reportada

Fase 2 - fronteiras e consolidacao

Cada bloco verifica a sua borda inferior e a direita

- Borda inferior: $(r1-1, c) \times (r1, c-1), (r1, c), (r1, c+1)$
- Borda direita: $(l, c1-1) \times (l-1, c1), (l, c1), (l+1, c1)$
- Reta + 2 diagonais = horizontal, vertical e diagonal
- Diagonais que cruzam a linha/coluna de corte tratam o ENCONTRO DE 4 BLOCOS
- Union-Find com compressao de caminho; menor rotulo vira raiz (deterministico)
- unir(): se raizes diferentes $\rightarrow$ une e unioes++ (dentro de `mtx_uf`)
- Verificar o mesmo par 2 vezes nao faz mal: so conta uniao efetiva

Exemplo rastreavel - Exemplo 3 (grade 2x2)



Componentes locais: A B | C F | D G | E H

Soma local = 8

Unioes efetivas: B-C, B-D, B-E = 3

(C-D, C-E, D-E: ja na mesma classe)

Objetos = 8 - 3 = 5 (esperado: 5)

```
$ ./conta-objetos-paralelo ex3_8x8.txt 4 2 2 -v
```

```
  Soma local: 8 | Unioes nas fronteiras: 3
```

```
Objetos: 5
```

Demonstracao

Terminal

```
$ make
$ ./conta-objetos-sequencial tests/obrigatorios/ex5_12x12.txt
$ ./conta-objetos-paralelo tests/obrigatorios/ex5_12x12.txt 9 3 3 -v
$ ./conta-objetos-paralelo tests/obrigatorios/ex3_8x8.txt 4 2 2 -v
$ make test
$ cat results/resumo_desempenho.md
```

Testes funcionais

make test - 8 configuracoes paralelas x 5 repeticoes por matriz

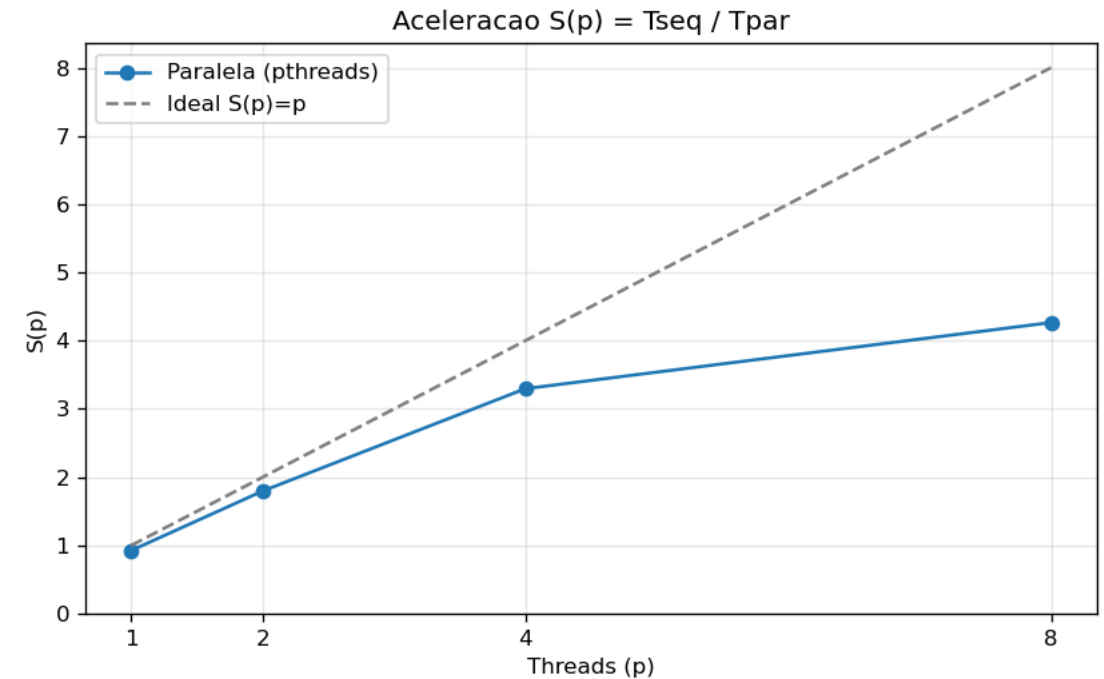
Ex.	Dim.	Esperado	Sequencial	Paralelo
1	5x5	3	3	3
2	6x8	4	4	4
3	8x8	5	5	5
4	9x12	6	6	6
5	12x12	7	7	7

- Adicionais: zeros (0), tudo 1 (1), xadrez so diagonal (1), X no encontro de 4 blocos (1), serpente (1), 25 pontos isolados (25), 1 linha (7)
- Configs: 1:1x1, 2:2x1, 2:1x2, 4:2x2, 4:3x3, 3:3x3, 8:4x4, 2:6x6
- 40 matrizes aleatorias + ThreadSanitizer: tudo identico

Desempenho

Matriz 4000x4000, 45% de 1s, mediana de 5 repeticoes (make bench)

Versao	p	T (ms)	S(p)	E(p)
Sequencial	1	309.5	1.00	1.00
Paralela	1	334.4	0.93	0.93
Paralela	2	172.2	1.80	0.90
Paralela	4	93.8	3.30	0.82
Paralela	8	72.5	4.27	0.53



Analise dos resultados

- Matrizes pequenas: paralelo mais lento - criar threads custa mais que o trabalho
- Paralelo com 1 thread < sequencial: rotulo[] e pai[] usam int (4 B) contra 1 B do visitado -> mais trafego de memoria; 2 fases + criacao de threads
- Ganho abaixo do ideal (Amdahl): leitura, joins e soma final sao sequenciais; algoritmo limitado por largura de banda de memoria
- Apple Silicon: nucleos P e E - acima do numero de nucleos P a eficiencia cai
- Consolidacao barata: $O(\text{perimetro dos blocos}) \ll O(L \times C)$; pouca disputa em mtx_uf

Conclusões

- Sequencial e paralelo sempre iguais: 5 obrigatorias, 7 adicionais, 8 configuracoes, 5 repeticoes
- Chave da correcao: rotulos globalmente unicos + Union-Find com regio critica minima
- Paralelismo efetivo: o flood fill (parte cara) roda simultaneamente nas threads
- Aprendizado: paralelizar = dividir dados + identificar estado compartilhado + projetar a consolidacao
- Melhoria futura: equivalencias locais por thread e reducao sem lock